

ANALISIS ALGORITHMA DAN PENGEMBANGAN PERANGKAT LUNAK

TUGAS KULIAH

disusun oleh

Markani

25201201010



**PROGRAM STUDI MAGISTER ILMU KOMPUTER
UNIVERSITAS TEKNOLOGI AKBA
MAKASSAR
2025**

1 Pendahuluan

Tugas Kuliah disusun sesuai dengan arahan yang diberikan oleh dosen pemateri perkuliahan, dimana tugas yang diberikan sebagai berikut

1. Memilih dua dari beberapa algoritma pengurutan terkecuali insertion sort.
2. menimplementasikan algoritma tersebut dalam sebuah bahasa pemrograman tanpa menggunakan library yang sudah ada.
3. Melakukan analisis terhadap algoritma yang telah diimplementasikan. dengan cara mengukur waktu eksekusi dari algoritma tersebut pada berbagai ukuran data yang berbeda (1000, 10000, 50000).
4. mencatat hasil tersebut kedalam tabel

penulis menggunakan bahasa pemrograman php 8.4 untuk mengimplementasikan algoritma pengurutan yang telah dipilih yaitu insertion sort dan quick sort. dan penulis telah menyimpan kode program pada repository github dengan alamat

https://github.com/markaniph5/markani_s2_unitama

dimana hal ini bertujuan untuk memudahkan dosen pemateri dalam melakukan review terhadap kode program yang telah diimplementasikan dan mengikuti etika penelitian yaitu reproduceable. untuk menguji algoritma yang telah diimplementasikan penulis menggunakan data acak dengan berbagai ukuran data yaitu 1000, 10000, dan 50000. menggunakan PHP unit Testing untuk mengukur waktu eksekusi dari algoritma yang telah diimplementasikan. perangkat keras yang penulis gunakan untuk menguji algoritma yang telah diimplementasikan adalah sebagai berikut :

```

C:\Users\User\Sites\markani_s2_unitama\analisis_alpro_pengembangan_perangkat_lunak\tugas1(main -> origin)
λ fastfetch
//////////////////////////////////// User@DESKTOP-2QEGQ1N
//////////////////////////////////// -----
//////////////////////////////////// OS: Windows 11 Pro (24H2) x86_64
//////////////////////////////////// Host: 20AWS2200D (ThinkPad T440p)
//////////////////////////////////// Kernel: WIN32_NT 10.0.26100.6899
//////////////////////////////////// Uptime: 13 mins
//////////////////////////////////// Shell: CMD (with Clink 1.2.46.0) 10.0.26100.6725
//////////////////////////////////// Display (Mi Monitor): 3440x1440 in 34", 50 Hz [External]
//////////////////////////////////// Display (BOE0714): 1920x1080 in 14", 60 Hz [Built-in] *
//////////////////////////////////// WM: Desktop Window Manager 10.0.26100.6899
//////////////////////////////////// WM Theme: Custom - #680081 (System: Dark, Apps: Dark)
//////////////////////////////////// Theme: Fluent
//////////////////////////////////// Icons: Recycle Bin
//////////////////////////////////// Font: Segoe UI (12pt) [Caption / Menu / Message / Status]
//////////////////////////////////// Cursor: Windows Default (32px)
//////////////////////////////////// Terminal: ConEmu64 210912
//////////////////////////////////// Terminal Font: Consolas (16pt)
//////////////////////////////////// CPU: Intel(R) Core(TM) i5-4210M (4) @ 2.60 GHz
//////////////////////////////////// GPU: Intel(R) HD Graphics 4600 (112.50 MiB) [Integrated]
//////////////////////////////////// Memory: 5.94 GiB / 15.88 GiB (37%)
//////////////////////////////////// Swap: 0 B / 2.88 GiB (0%)
//////////////////////////////////// Disk (C:): 66.70 GiB / 152.43 GiB (44%) - NTFS
//////////////////////////////////// Disk (D:): 110.82 MiB / 292.97 GiB (0%) - NTFS
//////////////////////////////////// Local IP (Wi-Fi): 10.239.154.48/24
//////////////////////////////////// Battery (Batt Device Name): 100% [AC Connected]
//////////////////////////////////// Locale: en-US
////////////////////////////////////
//////////////////////////////////// ██████████
C:\Users\User\Sites\markani_s2_unitama\analisis_alpro_pengembangan_perangkat_lunak\tugas1(main -> origin)
λ |

```

Spesifikasi Perangkat Keras

2 Pembahasan

2.1 Algoritma Pengurutan

Algoritma pengurutan adalah metode atau prosedur yang digunakan untuk mengatur elemen- elemen dalam suatu kumpulan data sehingga mereka berada dalam urutan tertentu. Urutan ini bisa berdasarkan nilai numerik, abjad, atau kriteria lainnya. Algoritma pengurutan sangat penting dalam ilmu komputer karena mereka digunakan dalam berbagai aplikasi, mulai dari pengolahan data hingga pencarian informasi. ada pun algorithma yang penulis pilih adalah sebagai berikut :

```
1      <?php
2          declare(strict_types=1);
3          namespace Markani\Tugas1\App;
4
5          class Algorithm {
6              public function quickSort(array $arr): array {
7                  $n = count($arr);
8                  if ($n <= 1) {
9                      return $arr;
10                 }
11
12                 // choose pivot (middle element)
13                 $pivot = $arr[intdiv($n, 2)];
14
15                 $left = [];
16                 $equal = [];
17                 $right = [];
18
19                 foreach ($arr as $value) {
20                     if ($value < $pivot) {
21                         $left[] = $value;
22                     } elseif ($value > $pivot) {
23                         $right[] = $value;
24                     } else {
25                         $equal[] = $value;
26                     }
27                 }
28             }
29         }
```

```

28
29         return array_merge($this->quickSort($left),
30                               $equal, $this->quickSort($right));
31     }
32
33     public function insertionSort(array $arr): array {
34         $n = count($arr);
35         for ($i = 1; $i < $n; $i++) {
36             $key = $arr[$i];
37             $j = $i - 1;
38             while ($j >= 0 && $arr[$j] > $key) {
39                 $arr[$j + 1] = $arr[$j];
40                 $j--;
41             }
42             $arr[$j + 1] = $key;
43         }
44         return $arr;
45     }

```

2.2 Kode Pengujian

Kode pengujian yang penulis gunakan untuk mengukur waktu eksekusi dari algoritma yang telah diimplementasikan adalah sebagai berikut :

```

1      <?php
2          declare(strict_types=1);
3          use PHPUnit\Framework\TestCase;
4          use Markani\Tugas1\App\Algorithm;
5
6          final class InsertionShort1000DataTest extends
7              TestCase
8          {
9              public function testAlgoithmInsertionSortFirstCase
10                 (): void
11             {
12                 // generate 1000 data

```

```

12         $total_data = 1000;
13         $data = [];
14         for ($i = 0; $i < $total_data; $i++ ) {
15             $data[] = $i+1;
16         }
17         $test_data = $data;
18         shuffle($test_data);
19
20         // mendeklarasikan objek
21         $algorithm = new Algorithm();
22         $sorted_data = $algorithm->insertionSort(
23             $test_data);
24         $this->assertEquals($data[$total_data - 1],
25             $sorted_data[$total_data - 1]);
26     }
27 }

```

```

1 <?php
2     declare(strict_types=1);
3     use PHPUnit\Framework\TestCase;
4     use Markani\Tugas1\App\Algorithm;
5
6     final class InsertionShort10000DataTest extends
7         TestCase
8     {
9         public function testAlgorithmInsertionSortSecondCase
10             (): void
11         {
12             // generate data
13             $total_data = 10000;
14             $data = [];
15             for ($i = 0; $i < $total_data; $i++ ) {
16                 $data[] = $i+1;
17             }
18             $test_data = $data;
19             shuffle($test_data);

```

```

20         // mendeklarasikan objek
21         $algorithm = new Algorithm();
22         $sorted_data = $algorithm->insertionSort(
23             $test_data);
24         $this->assertEquals($data[$total_data - 1],
25             $sorted_data[$total_data - 1]);
26     }
27 }

```

```

1  <?php
2      declare(strict_types=1);
3      use PHPUnit\Framework\TestCase;
4      use Markani\Tugas1\App\Algorithm;
5
6      final class InsertionShort50000DataTest extends
7          TestCase
8      {
9
10         public function testAlgoithmInsertionSortThirdCase
11             (): void
12         {
13             // generate data
14             $total_data = 50000;
15             $data = [];
16             for ($i = 0; $i < $total_data; $i++ ) {
17                 $data[] = $i+1;
18             }
19             $test_data = $data;
20             shuffle($test_data);
21
22             // mendeklarasikan objek
23             $algorithm = new Algorithm();
24             $sorted_data = $algorithm->insertionSort(
25                 $test_data);
26             $this->assertEquals($data[$total_data - 1],
27                 $sorted_data[$total_data - 1]);
28         }
29     }

```

```

1      <?php
2          declare(strict_types=1);
3          use PHPUnit\Framework\TestCase;
4          use Markani\Tugas1\App\Algorithm;
5
6          final class QuickShort1000DataTest extends TestCase
7          {
8
9              public function testAlgoithmQuickSortFirstCase():
10                 void
11             {
12                 // generate 1000 data
13                 $total_data = 1000;
14                 $data = [];
15                 for ($i = 0; $i < $total_data; $i++ ) {
16                     $data[] = $i+1;
17                 }
18                 $test_data = $data;
19                 shuffle($test_data);
20
21                 // mendeklarasikan objek
22                 $algorithm = new Algorithm();
23                 $sorted_data = $algorithm->quickSort(
24                     $test_data);
25                 $this->assertEquals($data[$total_data - 1],
26                     $sorted_data[$total_data - 1]);
27             }
28         }

```

```

1      <?php
2          declare(strict_types=1);
3          use PHPUnit\Framework\TestCase;
4          use Markani\Tugas1\App\Algorithm;
5
6          final class QuickShort10000DataTest extends TestCase
7          {
8
9              public function testAlgoithmQuickSortSecondCase():

```



```

10         void
11     {
12         // generate data
13         $total_data = 10000;
14         $data = [];
15         for ($i = 0; $i < $total_data; $i++ ) {
16             $data[] = $i+1;
17         }
18         $test_data = $data;
19         shuffle($test_data);
20
21         // mendeklarasikan objek
22         $algorithm = new Algorithm();
23         $sorted_data = $algorithm->quickSort(
24             $test_data);
25         $this->assertEquals($data[$total_data - 1],
26             $sorted_data[$total_data - 1]);
27     }
28 }

```